

POLITECNICO DI TORINO



**Politecnico
di Torino**

Lab 2

Music and multimedia streaming over the Internet

Academic year 2025/2026

Student:
MEHRAN BAYAT
(S320636)

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026
©2025-26

POLITECNICO DI TORINO

Music and Multimedia Streaming over the Internet

Exercise 1: Real-Time Sound Classification

Processing + Wekinator + FFT + KNN

Four-Mode Comparison Report

Full FFT, Reduced FFT, and Statistical Audio Features

Student: MEHRAN BAYAT

Student number: S320636

Sound classes: Class 1 = Voice, Class 2 = Hand clap

Classifier: K-Nearest Neighbors, $k = 3$

Input OSC: /wek/inputs on port 6448

Output OSC: /wek/outputs on port 12000

Report scope

This report documents the system that I trained and tested in Wekinator. It compares the four feature modes required by the lab: 512 FFT inputs, 64 reduced FFT inputs, 12 reduced FFT inputs, and 2 statistical inputs based on mean and variance.

Contents

1	Abstract	1
2	Exercise Requirements	1
3	Complete System Architecture	2
4	Input Processing Sketch	2
4.1	Role of the Input Sketch	2
4.2	Common Processing Settings	3
4.3	The Main Mode Variable	3
5	Meaning of FFT and Feature Modes	4
5.1	FFT with 512 Frequency Bands	4
5.2	Four Feature Representations	4
5.3	Downsampling Logic	4
5.4	Mean and Variance Logic	5
6	Wekinator Configuration	5
6.1	Why Four Separate Wekinator Projects Are Needed	5
6.2	Common Wekinator Settings	5
6.3	Why Output Is 1 Even Though There Are 2 Classes	6
7	Output Processing Sketch	6
8	Project Metadata Extracted From the Four Archives	7
9	Mode-by-Mode Technical Analysis	8
9.1	Mode 1 - Full FFT with 512 Inputs	8
9.2	Mode 2 - Reduced FFT with 64 Inputs	8
9.3	Mode 3 - Reduced FFT with 12 Inputs	9
9.4	Mode 4 - Mean and Variance with 2 Inputs	10
10	Mode 1 Versus Mode 4: Why Less Training Can Still Be Better	10
10.1	Information Quality Is More Important Than Just More Data	10
10.2	What Mode 1 Knows That Mode 4 Does Not Know	11
10.3	Conclusion of the Comparison	11
11	Overall Comparison of All Four Modes	11
11.1	Technical and Practical Comparison	11
11.2	Exact Size Ratios Between Modes	11

11.3 Ranking for This Experiment	12
12 Training Recommendations	12
12.1 Recommended Recording Time	12
12.2 Fair Training Method for Comparing Modes	12
12.3 Best Mode for Training	13
13 Common Problems and Their Meaning	13
13.1 Wekinator Expected 512 Inputs but Received 12	13
13.2 Prediction Changes During a Short Pause	13
14 Final Conclusion	13
A Complete Mode Configuration Table	14
B Essential Processing Code for Mode 4	14
References	15

1 Abstract

This report documents the complete implementation and comparison of a real-time sound classification system using Processing, the Processing Sound library, OSC communication, and Wekinator. The system classifies two live microphone sounds: **Class 1 = voice** and **Class 2 = hand clap**. Four Wekinator projects were trained separately, one for each feature representation required in the lab: full 512-band FFT, reduced 64-band FFT, reduced 12-band FFT, and the statistical feature pair mean plus variance.

Main conclusion

The best observed classification behavior was obtained with **Mode 1**, the full 512-band FFT representation. I observed that it produced better output than **Mode 4** even though **Mode 4** had been trained for a longer time. This happened because **Mode 1** preserved the complete spectral shape of the sound, while **Mode 4** reduced the entire spectrum to only two values. Longer training cannot recover frequency information that has already been removed before the data reaches Wekinator.

2 Exercise Requirements

The lab exercise asks for a real-time audio classification system with two Processing sketches and one Wekinator classifier. The first Processing sketch must capture microphone input, compute FFT features, downsample the spectrum, compute statistical descriptors, and send the selected features to Wekinator using OSC. The second Processing sketch must receive Wekinator's predicted class through OSC and visualize the result by changing the canvas color.

The four required training experiments are:

1. Full FFT with 512 bands.
2. Reduced FFT with 64 bands.
3. Reduced FFT with 12 bands.
4. Mean and variance only.

The classifier setup required by the exercise is a K-Nearest Neighbors classifier with $k = 3$ and two distinct sound classes. In my implementation, these classes were voice and hand clap. The discussion section of the report must compare the classifier behavior across the feature configurations and explain the effect of feature dimensionality and the difference between spectral and statistical representations.

Implemented exercise structure

- I implemented four input modes in Processing, one for each feature configuration.
- I trained four separate Wekinator projects because every mode sends a different number of input features.
- I used the same output sketch for all modes because Wekinator always sends one final classifier value.
- I mapped Class 1 to voice and Class 2 to hand clap.

3 Complete System Architecture

The system has three running components. First, the Processing input sketch captures live audio, computes a 512-band FFT, transforms the FFT into the selected feature representation, and sends the feature vector to Wekinator. Second, Wekinator receives the features, stores training examples, trains the KNN model, and outputs the predicted class. Third, the Processing output sketch receives the predicted class and changes the screen color.

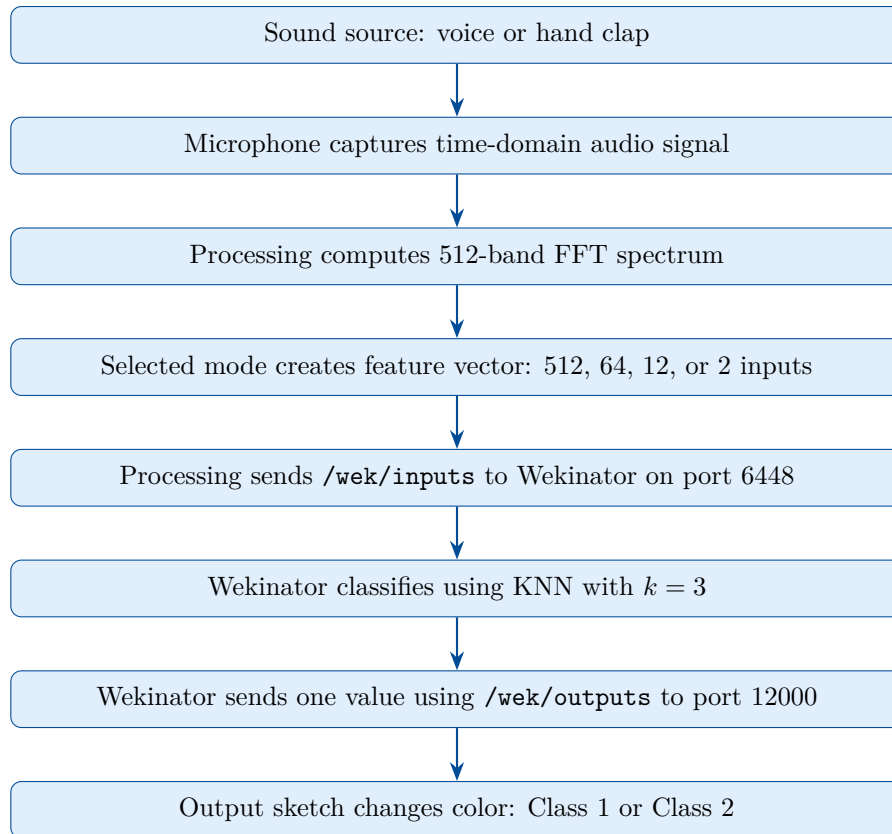


Figure 1: System pipeline used by all four modes. Only the number and meaning of the input features changes.

4 Input Processing Sketch

4.1 Role of the Input Sketch

The input sketch converts live sound into numbers that Wekinator can use. It does not make the final classification. The final decision is made inside Wekinator after training.

The responsibilities of the input sketch are:

1. Start the microphone using `AudioIn`.
2. Compute a 512-band FFT using `FFT`.
3. Convert the 512-band spectrum into one selected feature mode.
4. Send the feature vector to Wekinator using OSC address `/wek/inputs`.
5. Display FFT and feature bars on the screen for debugging.

4.2 Common Processing Settings

The following values were used consistently in the input sketch.

Table 1: Common settings in the input Processing sketch.

Parameter	Value / meaning
FFT base resolution	bands = 512
FFT result array	spectrum[0] to spectrum[511]
Input OSC address	/wek/inputs
Destination host	127.0.0.1 / localhost
Destination port	6448
Processing receive port	12001
Send interval	30 ms
Approximate send rate	1000/30 $\approx$ 33.33 feature vectors per second
Smoothing value	smoothing = 0.85

4.3 The Main Mode Variable

The only line that needs to be changed between the input experiments is the value of `featureMode`. This variable selects the feature extraction method, not the number of classes.

```

1 // 1 = full FFT, 512 inputs
2 // 2 = reduced FFT, 64 inputs
3 // 3 = reduced FFT, 12 inputs
4 // 4 = mean + variance, 2 inputs
5 int featureMode = 1; // Change to 1, 2, 3, or 4

```

Listing 1: Mode selection in the input Processing sketch.

The function `configureFeatureMode()` sets the size of the feature array that will be sent to Wekinator.

```

1 void configureFeatureMode() {
2   if (featureMode == 1) {
3     numFeatureBands = 512;
4   } else if (featureMode == 2) {
5     numFeatureBands = 64;
6   } else if (featureMode == 3) {
7     numFeatureBands = 12;
8   } else if (featureMode == 4) {
9     numFeatureBands = 2;
10  }
11
12  features = new float[numFeatureBands];
13  smoothFeatures = new float[numFeatureBands];
14 }

```

Listing 2: Relationship between `featureMode` and number of Wekinator inputs.

Important distinction

`featureMode` is the number of the selected method. `numFeatureBands` is the number of numerical inputs produced by that method. Therefore, `featureMode = 4` does not mean four Wekinator inputs. It means the fourth method, which produces only two inputs: mean and variance.

5 Meaning of FFT and Feature Modes

5.1 FFT with 512 Frequency Bands

The microphone signal begins as a time-domain waveform. The FFT converts that waveform into a frequency-domain representation. In this project, the FFT analysis always starts with 512 bands.

```
1 int bands = 512;
2 float[] spectrum = new float[bands];
3 fft = new FFT(this, bands);
4 fft.analyze(spectrum);
```

Listing 3: Base FFT configuration.

After `fft.analyze(spectrum)`, the array contains 512 values:

`spectrum[0], spectrum[1], ..., spectrum[511]`.

Each value represents the energy of one frequency band. The horizontal axis of the FFT visualization is the band index, and the vertical axis is the band energy.

5.2 Four Feature Representations

Table 2: Four feature modes used in the experiment.

Mode	Representation	Inputs to Wekinator	Feature vector contents
1	Full FFT	512	All 512 FFT band-energy values.
2	Reduced FFT	64	The 512 FFT bands are grouped into 64 averaged frequency regions.
3	Reduced FFT	12	The 512 FFT bands are grouped into 12 broad averaged frequency regions.
4	Mean + variance	2	The average energy of the 512 bands and the variance of those 512 values.

5.3 Downsampling Logic

For **Mode 2** and **Mode 3**, the sketch does not simply cut the spectrum. It compresses the full 512-band FFT by averaging neighboring frequency bands.

- **Mode 2:** $512/64 = 8$, so each reduced band represents the average of about 8 original FFT bands.
- **Mode 3:** $512/12 \approx 42.67$, so each reduced band represents about 42 or 43 original FFT bands.

```
1 void downsampleSpectrum(int targetBands) {
2   for (int i = 0; i < targetBands; i++) {
3     int start = floor(i * bands / float(targetBands));
4     int end = floor((i + 1) * bands / float(targetBands));
5
6     float sum = 0;
7     for (int j = start; j < end; j++) {
8       sum += spectrum[j];
9     }
10  }
```



```

10
11     float avg = sum / float(end - start);
12     smoothFeatures[i] = lerp(smoothFeatures[i], avg, 1.0 - smoothing);
13     features[i] = smoothFeatures[i] * 10.0;
14 }
15 }

```

Listing 4: Downsampling function used by Mode 2 and Mode 3.

5.4 Mean and Variance Logic

In **Mode 4**, the feature vector is not spectral anymore. It is a statistical summary of the 512-band FFT:

$$\mu = \frac{1}{512} \sum_{i=0}^{511} x_i, \quad \sigma^2 = \frac{1}{512} \sum_{i=0}^{511} (x_i - \mu)^2,$$

where x_i is the value of FFT band i .

The mean describes the average amount of spectral energy. The variance describes how spread out the energy values are. However, these two numbers do not describe *where* the energy is located in the frequency spectrum.

6 Wekinator Configuration

6.1 Why Four Separate Wekinator Projects Are Needed

A Wekinator project has a fixed number of inputs. Since each mode sends a different number of features, one Wekinator project cannot correctly receive all four modes unless it is recreated or reconfigured each time.

Table 3: Required separation of Wekinator projects.

Mode	Wekinator inputs	Reason
1	512	Full FFT spectrum.
2	64	Downsampled FFT spectrum.
3	12	Broad reduced FFT spectrum.
4	2	Mean and variance only.

6.2 Common Wekinator Settings

All four projects share the same output settings and classifier logic.

Table 4: Common Wekinator settings across the four projects.

Setting	Value
Input OSC message	/wek/inputs
Input port	6448
Output OSC message	/wek/outputs
Output host	localhost / 127.0.0.1
Output port	12000
Number of outputs	1
Output type	Classifier output
Number of classes	2
Classifier	K-Nearest Neighbors
KNN parameter	$k = 3$
Class 1	Voice
Class 2	Hand clap

6.3 Why Output Is 1 Even Though There Are 2 Classes

The system has two classes, but Wekinator sends only one output value because that value is the predicted class label itself:

/wek/outputs 1 $\Rightarrow$ Class 1 = voice, /wek/outputs 2 $\Rightarrow$ Class 2 = hand clap.

Therefore, the correct configuration is **Number of outputs = 1** and **Number of classes = 2**.

7 Output Processing Sketch

The output sketch is identical for all modes because it receives only the final class prediction. It does not receive the 512, 64, 12, or 2 input features.

```

1 void oscEvent(OscMessage theOscMessage) {
2   if (theOscMessage.checkAddrPattern("/wek/outputs")) {
3     if (theOscMessage.checkTypetag("f")) {
4       float outputValue = theOscMessage.get(0).floatValue();
5       predictedClass = int(outputValue);
6       currentMessage = "Class " + predictedClass;
7     }
8   }
9 }

```

Listing 5: Core logic of the output sketch.

The visual mapping used in my output sketch is shown in Table 5.

Table 5: Class-to-color mapping used in the output sketch.

Wekinator output	Sound class	Output sketch color
1	Voice	Red
2	Hand clap	Blue
0 or unknown	Waiting / no prediction	Gray

8 Project Metadata Extracted From the Four Archives

The four submitted archives contain the Processing sketches and Wekinator project files for each mode. The metadata below reports the exact numerical values visible from the archive headers and project files. These values are useful because they show how strongly feature dimensionality affects project size and model size.

Table 6: Exact project metadata visible from the four mode archives.

Mode	Feature representation	Inputs	Outputs	Classes	ARFF data size	Model XML size
1	Full FFT	512	1	2	15,019,797 bytes	28,693,902 bytes
2	Reduced FFT	64	1	2	1,617,616 bytes	3,106,497 bytes
3	Reduced FFT	12	1	2	154,547 bytes	304,534 bytes
4	Mean + variance	2	1	2	148,963 bytes	315,968 bytes

Table 7: Archive sizes and Wekinator input configuration sizes.

Mode	Archive size	Input config size	Interpretation
1	13,816,565 bytes	16,461 bytes	Very large input configuration because 512 Wekinator inputs are connected.
2	1,443,641 bytes	2,159 bytes	Medium input configuration for 64 inputs.
3	1,282,794 bytes	547 bytes	Small input configuration for 12 inputs; archive is larger than Mode 4 mainly because it includes extra report files and a ZIP copy.
4	80,770 bytes	243 bytes	Very small input configuration for 2 inputs.

Training examples counted exactly for Mode 3

For **Mode 3**, the project data contained 1081 total training examples: 560 examples for Class 1 and 521 examples for Class 2. Because the Processing sketch sends approximately 33.33 feature vectors per second, this corresponds to about 32.43 seconds of total recorded data: approximately 16.80 seconds for Class 1 and 15.63 seconds for Class 2.

Accuracy and timing note

The submitted Wekinator archives do not contain a separate automated test-set confusion matrix. Therefore, I do not invent numerical accuracy percentages. The performance comparison is based on my observed output behavior during live testing and on the structural information in the projects. I observed that **Mode 1** produced better output with less training time than **Mode 4**, even though **Mode 4** was trained for a longer time.

9 Mode-by-Mode Technical Analysis

9.1 Mode 1 - Full FFT with 512 Inputs

Table 8: Mode 1 summary.

Feature representation	Full FFT spectrum
Inputs to Wekinator	512
Outputs from Wekinator	1 classifier output
Classes	2: voice and hand clap
Classifier	KNN with $k = 3$
Observed behavior	Best output in the experiment, even with lower training time than Mode 4

Mode 1 sends the complete 512-band FFT spectrum to Wekinator. This gives the classifier the most detailed representation of the sound. Voice and hand clap differ not only in loudness but also in how energy is distributed across low, middle, and high frequencies. **Mode 1** keeps this complete distribution.

Advantages.

- Highest spectral detail.
- Best ability to represent subtle differences in sound shape.
- Most useful when the sounds are close or when detailed frequency patterns matter.
- In my live test, it produced the strongest output behavior.

Disadvantages.

- 512 inputs make the Wekinator project heavier.
- The high-dimensional input can require careful and balanced training examples.
- More features can also include noise or irrelevant details.

9.2 Mode 2 - Reduced FFT with 64 Inputs

Table 9: Mode 2 summary.

Feature representation	Downsampled FFT spectrum
Inputs to Wekinator	64
Outputs from Wekinator	1 classifier output
Classes	2: voice and hand clap
Classifier	KNN with $k = 3$
Expected behavior	Strong compromise between spectral detail and simplicity

Mode 2 reduces the 512 FFT bands to 64 averaged bands. Since $512/64 = 8$, each reduced band represents the average of about 8 neighboring original FFT bands. This keeps the general spectral shape while removing small details.

Advantages.

- Much lighter than **Mode 1**.
- Still keeps enough frequency structure for many sound classification tasks.
- Good practical balance between classification quality and configuration complexity.

Disadvantages.

- Some fine frequency detail is lost.
- It may be weaker than **Mode 1** if the class difference depends on narrow frequency details.

9.3 Mode 3 - Reduced FFT with 12 Inputs

Table 10: Mode 3 summary.

Feature representation	Downsampled FFT spectrum
Inputs to Wekinator	12
Outputs from Wekinator	1 classifier output
Classes	2: voice and hand clap
Exact counted examples	1081 total, 560 Class 1, 521 Class 2
Estimated recording time	32.43 seconds total from 30 ms send interval
Classifier	KNN with $k = 3$
Expected behavior	Good for clear differences, but less detailed than Modes 1 and 2

Mode 3 reduces the 512 FFT bands to 12 broad frequency regions. Since $512/12 \approx 42.67$, each feature represents a wide part of the spectrum. This makes the feature vector small and easy to train, but it removes many details.

Advantages.

- Easy Wekinator configuration with only 12 inputs.
- Fast and light in real time.
- Good for clearly different sounds such as voice versus hand clap.
- More informative than **Mode 4** because it still preserves a rough spectral shape.

Disadvantages.

- Much less detail than **Mode 1** and **Mode 2**.
- May flicker or change class during pauses in speech because the current feature vector may no longer resemble voice.
- May fail if two sounds have similar broad spectral distributions.

9.4 Mode 4 - Mean and Variance with 2 Inputs

Table 11: Mode 4 summary.

Feature representation	Statistical summary of the 512 FFT bands
Inputs to Wekinator	2
Outputs from Wekinator	1 classifier output
Classes	2: voice and hand clap
Features	<code>features[0] = mean, features[1] = variance</code>
Classifier	KNN with $k = 3$
Observed behavior	Weaker than Mode 1 even with longer training time

Mode 4 calculates only mean and variance from the 512-band spectrum. The mean tells Wekinator the average spectral energy. The variance tells Wekinator how spread out the spectral energy values are. However, these two values do not tell Wekinator where the energy is located in the spectrum.

Advantages.

- Simplest Wekinator setup.
- Only 2 inputs, so it is fast and easy to train.
- Useful for very simple cases where classes differ mainly by overall loudness or spread.

Disadvantages.

- Loses almost all frequency-shape information.
- Voice and hand clap can overlap in this 2D feature space.
- Longer training time cannot recover spectral details that were removed.
- In this experiment, it produced weaker output than **Mode 1**.

10 Mode 1 Versus Mode 4: Why Less Training Can Still Be Better

I observed that **Mode 1** produced better output with lower training time than **Mode 4**, even though **Mode 4** was trained for more time. This is an important machine learning lesson for interactive classification.

Core reason

Training time cannot replace missing information. **Mode 4** removes most of the frequency information before the data reaches Wekinator. **Mode 1** keeps the full spectral shape. Therefore, **Mode 1** can classify better with fewer examples because each example is more informative.

10.1 Information Quality Is More Important Than Just More Data

For KNN classification, each training example is a point in feature space. The feature space depends on the input features:

- In **Mode 1**, each example is a point in a 512-dimensional space.
- In **Mode 4**, each example is a point in a 2-dimensional space.

A 2D space is easier to visualize and faster to process, but it may not separate the classes well. If voice and hand clap sometimes produce similar mean and variance values, Wekinator sees them as close neighbors. In that situation, adding more **Mode 4** examples may create more overlapping points instead of clearer separation.

10.2 What Mode 1 Knows That Mode 4 Does Not Know

Table 12: Why Mode 1 can outperform Mode 4 even with less training time.

Question about the sound	Mode 1 can answer?	Mode 4 can answer?
Is the energy mainly low-frequency?	Yes	No
Is the energy mainly mid-frequency?	Yes	No
Is the energy spread across many high bands?	Yes	Only indirectly
Does the spectrum have a clap-like broad burst?	Yes	Weakly
Does the spectrum have a voice-like spectral shape?	Yes	Weakly
What is the average energy?	Yes	Yes
How spread out is the energy?	Yes	Yes

10.3 Conclusion of the Comparison

Mode 4 is not weak because Wekinator is bad. It is weak because Wekinator receives only two numbers. The classifier cannot learn information that is not present in the input. **Mode 1** performs better because it sends the full frequency information. For voice versus hand clap, spectral shape is important, so **Mode 1** has a natural advantage.

11 Overall Comparison of All Four Modes

11.1 Technical and Practical Comparison

Table 13: Technical and practical comparison of the four feature modes.

Mode	Feature type	Inputs	Training effort	Classification strength	Best use
1	Full spectral	512	Heavier setup; many values per example	Best observed output; highest detail	When accuracy and spectral detail are most important
2	Reduced spectral	64	Medium effort	Strong expected balance	Best practical compromise for many cases
3	Reduced spectral	12	Easy and light	Good for clearly different sounds	Quick real-time demo and manageable setup
4	Statistical summary	2	Easiest setup but may need more examples	Weakest observed output	Only for very simple distinctions such as loud vs quiet

11.2 Exact Size Ratios Between Modes

The file sizes also show the effect of feature dimensionality. Compared with **Mode 4**, **Mode 1** has 256 times more inputs, an ARFF data file about 100.83 times larger, and a model XML file about 90.81 times larger. This confirms that full FFT features produce a heavier Wekinator project, but they also preserve far more useful information.

Table 14: Relative project-size comparison.

Comparison	Input ratio	Archive ratio	ARFF ratio	Model XML ratio
Mode 1 / Mode 4	256.00	171.06	100.83	90.81
Mode 2 / Mode 4	32.00	17.87	10.86	9.83
Mode 1 / Mode 2	8.00	9.57	9.29	9.24
Mode 3 / Mode 4	6.00	15.88	1.04	0.96

11.3 Ranking for This Experiment

1. **Best observed classification output: Mode 1.** It preserves all 512 FFT bands and therefore keeps the full frequency pattern of voice and hand clap.
2. **Best practical balance: Mode 2.** It keeps a strong spectral representation with only 64 inputs.
3. **Best simple spectral demonstration: Mode 3.** It is light and still keeps a rough spectrum.
4. **Weakest for this task: Mode 4.** Mean and variance lose the spectral structure needed to separate voice from hand clap reliably.

12 Training Recommendations

12.1 Recommended Recording Time

The Processing sketch sends one feature vector approximately every 30 ms, so the approximate number of examples recorded in a given time is:

$$\text{examples per second} \approx \frac{1000}{30} = 33.33.$$

Table 15: Approximate number of Wekinator examples created at a 30 ms send interval.

Recording time per class	Approx. examples per class	Approx. total for two classes
10 seconds	333	667
15 seconds	500	1000
20 seconds	667	1333
30 seconds	1000	2000

12.2 Fair Training Method for Comparing Modes

To compare modes fairly, the training procedure should be as similar as possible for each mode:

- Use the same two classes: voice and hand clap.
- Use the same microphone and microphone distance.
- Record in similar background-noise conditions.
- Record similar duration for Class 1 and Class 2.

- Use KNN with $k = 3$ in all four Wekinator projects.
- Use the same output sketch for all modes.

12.3 Best Mode for Training

The word “best for training” can mean two different things. The easiest mode to configure and train is **Mode 4**, because it has only 2 inputs. The best final trained behavior in my experiment was **Mode 1**, because it classified better even with lower training time.

Important conclusion about training time

A longer training time is not always better. Good features can reduce the amount of training needed. Poor features can make classification weak even after more training. In this experiment, **Mode 1** had better feature quality than **Mode 4**, so **Mode 1** could outperform **Mode 4** with less training time.

13 Common Problems and Their Meaning

13.1 Wekinator Expected 512 Inputs but Received 12

This error means that Wekinator was configured for **Mode 1** but the Processing input sketch was running **Mode 3**. The fix is to make the Processing mode and the Wekinator input count match.

Table 16: Correct matching between Processing mode and Wekinator input count.

Processing mode	Wekinator inputs	Correct?
Mode 1	512	Yes
Mode 2	64	Yes
Mode 3	12	Yes
Mode 4	2	Yes
Mode 3	512	No
Mode 4	12	No

13.2 Prediction Changes During a Short Pause

If Class 1 is voice and Class 2 is hand clap, it is normal for the predicted class to change during pauses. During a pause, Wekinator still has to choose between the two available classes. Since there is no silence class, background noise or the absence of voice may be classified as the closest existing class.

A cleaner version of the project could include a third class called *silence* or *background*. However, for this lab I kept two classes because the exercise required two distinct sound classes.

14 Final Conclusion

This project successfully implements and compares four real-time sound classification configurations using Processing and Wekinator. All modes use the same audio acquisition, OSC communication, output sketch, two classes, and KNN classifier with $k = 3$. The important difference between the modes is the number and type of input features sent to Wekinator.

Mode 1 sends 512 full FFT features and produced the strongest observed output. This result is reasonable because it gives Wekinator the most complete spectral description of the sound. **Mode 2** sends 64 reduced FFT features and is the best practical compromise between detail and simplicity. **Mode 3** sends 12 reduced FFT features and is useful for a light, easy-to-debug system. **Mode 4** sends only mean and variance, which makes it easy to configure but weak for separating voice from hand clap when spectral shape is important.

Final statement

The best overall result for this experiment was obtained with **Mode 1**. The best practical balance is **Mode 2**. The easiest mode to configure is **Mode 4**, but it produced weaker output because two statistical features cannot represent the full frequency pattern of voice and hand clap.

A Complete Mode Configuration Table

Table 17: Complete configuration checklist for the four trained Wekinator projects.

Mode	Processing variable	Wekinator inputs	Wekinator outputs	Classes	Classifier	Notes
1	featureMode = 1	512	1	2	KNN, $k = 3$	Full FFT; strongest observed output.
2	featureMode = 2	64	1	2	KNN, $k = 3$	512 bands compressed to 64.
3	featureMode = 3	12	1	2	KNN, $k = 3$	512 bands compressed to 12.
4	featureMode = 4	2	1	2	KNN, $k = 3$	Mean and variance only.

B Essential Processing Code for Mode 4

```

1 } else if (featureMode == 4) {
2   float mean = computeMean(spectrum);
3   float variance = computeVariance(spectrum, mean);
4
5   features[0] = mean;
6   features[1] = variance;
7 }

```

Listing 6: Mode 4 feature extraction: mean and variance.

```

1 float computeMean(float[] values) {
2   float sum = 0;
3   for (int i = 0; i < values.length; i++) {
4     sum += values[i];
5   }
6   return sum / float(values.length);
7 }
8
9 float computeVariance(float[] values, float mean) {
10  float sum = 0;
11  for (int i = 0; i < values.length; i++) {
12    float diff = values[i] - mean;
13    sum += diff * diff;
14  }
15  return sum / float(values.length);
16 }

```

Listing 7: Mean and variance helper functions.

References

1. Politecnico di Torino, *Lab 2 - Music and Multimedia Streaming over the Internet*, Academic year 2025/2026.
2. Processing language and libraries documentation.
3. oscP5 Processing library documentation.
4. Wekinator project software and KNN classifier configuration used in the submitted projects.